DEPLOYMENT DOCUMENTATION

Step by step build on how to use and understand Team 5's AI-Powered Malware Analysis and Threat Hunting System.

DOWNLOAD / SETUP (GIT)

Git is used to download the codebase to manage the repository. If Git is not installed, run:

`sudo apt install -y git`

CLONE THE REPOSITORY

```
git clone https://github.com/RRougeX/Team5FinalProject
cd Team5FinalProject
ls #Confirm all was downloaded correctly
```

PYTHON VIRTUAL ENVIRONEMNT SETUP

A python virtual environment (.venv) isolates the projects packages from system level packages. This helps prevent dependency conflicts and smooths out the application.

SET UP THE VIRUTAL ENVIRONMENT

```
python -m venv .venv
source .venv/bin/activate
which python
```

This will create and activate your virtual environment. You should see a `(.venv)` appear at the beginner of your terminal prompt indicating you are in the virtual environment.

DEPENDENCIES / REQUIREMENTS.TXT

The system uses several system tools and Python packages for operation. They are needed for document processing, embeddings, vector storage, model communication, and more. All python packages should be installed in the systems virtual environment

SYSTEM DEPENDENCIES

How To Install the required Ubuntu packages:

```
sudo apt update
sudo apt install -y python3 python3-pip python-venv
sudo apt install -y git build-essential curl unzip jq
```

This will install general dependencies that will support development on the project

PYTHON DEPENDENCIES

Install all required Python packages with the requirements.txt file:

```
python -m pip install -r requirements.txt
```

Dependency	Purpose
Python 3	Backbone for the system
boto3	Communicates with AWS Bedrock and validates creds
chromadb	Vectored storing from chunks, embeddings, and metadata
Llama-index-core	Processes document indexing, retravel, and querying
Llama-index-vector-stores-chroma	Connects LlamaIndex to ChromaDB
Llama-index-llms-bedrock-converse	Connects Amazon Nova model to LlamaIndex query engine
Llama-index-readers-file	Loads and reads through malware reports from PDFs
Llama-index-embeddings-huggingface	Connects the local embedding model to LlamaIndex (BGE)
sentence-transformers	Loads and operate the local BGE embedding model
torch	Powers local embedding model
pypdf	Extracts text from PDF malware reports
pytest	Automated Python testing
ruff	Code formatting and syntax

DATA FOMATTING / STORAGE

The system separates original malware reports from the data generated during the knowledge-base creation. This prevents source documents changing and makes it easier to identify which information was supplied by the user and which information was generated by the system.

RAW DATA (DATA/RAW)

`data/raw` stores user inputted malware reports in PDF or txt format. Reports can be organized into subfolders by source or organization. Example: `data/raw/CISA/CISA_Malware_Report.txt`

PROCESSED DATA (DATA/PROCESSED)

`data/processed` stores extracted data, embeddings, metadata, and persistent ChromaDB files. These are all generated from the system using the `raw` folders malware reports.

REPORTS

`reports/` stores the final threat-hunting reports generated by the model and system (NOVA). Reports are saved in markdown format and should be reviewed after generating them to ensure the response is accurate and reliable.

PROJECT FILE STRUCTURE

```
Team5FinalProject/
│
├── data/
│   ├── raw/
│   │   └── [Original malware reports]
│   │
│   └── processed/
│       └── [Generated ChromaDB knowledge bases]
│
├── reports/
│   └── [Generated threat-hunting reports]
│
├── src/
│   ├── system/
│   │   ├── chromaDB.py
│   │   ├── indexReport.py
│   │   ├── models.py
│   │   └── queryManager.py
│   │
│   ├── tools/
│   │   ├── aws_key_manager.py
│   │   ├── folderPathLogic.py
│   │   ├── helper_functions.py
│   │   ├── logCleanup.py
│   │   └── title.py
│   │
│   └── main.py
│
├── config.ini
├── .gitignore
├── README.md
└── requirements.txt
```

DEPLOYMENT DOCUMENTATION

MAIN.PY – MAIN APPLICATION

Main.py is where the system is tied together to execute. You will run this file to start the system.

```
Python3 main.py
```

IMPORTS

```
import sys
from tools.aws_key_manager import enter_new_keys, is_AWS_config_credentials_valid
from tools.title import header
from system.chromaDB import load_embedding_data, create_embedding_data
from tools.log_cleanup import cleanLogs
from tools.helper_functions import options_printer
from system.queryManager import start_query_manager
```

This will connect `main.py` to the rest of the project. The tools folder will provide our title, menus, log cleanup, and AWS credentials management. The system folder will handle ChromaDB and its embeddings, along with Amazon NOVAs query interface

LOADING AND CREATING EMBEDDING DATA

```
def load_or_create_embedding_data():
    user_input = options_printer([
        "Load existing embedding data",
        "Create new embedding data",
        "Back"
    ])

    match user_input:
        case "1":
            print("Loading existing embedding data.")
            return load_embedding_data()

        case "2":
            print("Creating new embedding data.")
            return create_embedding_data()

        case "3":
            return None
```

This method will allow the users to load an existing ChromaDB knowledge base or create a new embedding data set for their new malware reports. Back returns them to the main menu

AWS CREDENTIAL MENU

```
def awsKeyCheck():
    user_input = options_printer([
        "Enter new keys",
        "Check if keys are valid",
        "Back (keep using current keys)"
    ], "AWS Keys")

    match user_input:
        case "1":
            enter_new_keys()

        case "2":
            valid = is_AWS_config_credentials_valid()
            print(f"Keys are {'Valid' if valid else 'Invalid'}")

        case "3":
            return
```

This method allows users to enter their Pluralsight AWS credentials or verify if their currently stored credentials are still valid and useable (Checks Pluralsight console status using boto3)

MAIN APPLICATOIN STARTUP

```
def main():
    cleanLogs()
    print(header())
    print("Welcome to Team 5 Malware RAG!")

    while True:
        user_input = options_printer([
            "Change AWS bedrock AI keys",
            "Load or create embedding data and run a model",
            "Exit"
        ], "Select a option.")
```

Creates the `main()` methods that cleans old logs, displays the title, and opens a menu that will be used throughout the system. (Until the session is killed or user enters “exit”)

MAIN MENU ACTIONS

```
        match user_input:
            case "1":
                awsKeyCheck()

            case "2":
                if not is_AWS_config_credentials_valid():
                    print("Please fix your AWS keys in option 1.")
                    continue

                index = load_or_create_embedding_data()

                if index == None:
                    continue

                start_query_manager(index)

            case "3":
                print("Bye!")
                sys.exit(0)
```

Optoin1 opens the credential manager. Option 2 continues to the main application. It will check to ensure the credentials entered are validated. Option 3 safely closes the application.

TOOLS – HELPER_FUNCTIONS.PY

Helper_functions.py contains reusable functions that are used across the system.

MENU FUNCTION

```
def options_printer(
    options_text=["!!!NO OPTIONS INPUTED!!!"],
    question="What would you like to do?",
    user_input=""
):
    """
    Takes in a list of strings that will be displayed as
    numbered options. Returns the user's selected option.
    """

    if len(options_text) <= 0:
        return ""

    options = []
```

```
    print(f"\n{question}")

    for i, option in enumerate(options_text, start=1):
        print(f"{i}. {option}")
        options.append(str(i))
```

This will connect `main.py` to the rest of the project. The tools folder will provide our title, menus, log cleanup, and AWS credentials management. The system folder will handle ChromaDB and its embeddings, along with Amazon NOVAs query interface

INPUT VALIDATION

```
    while True:
        user_input = input(
            f"Enter your choice: {choices_str}\n: "
        ).strip()

        if user_input not in options:
            print(
                f"Invalid choice. Please enter a valid "
                f"option number: {choices_str}"
            )
            continue

        return user_input
```

Loops in user input until the user enters a choice. Provides an error message for an invalid input.

TOOLS – LOG_CLEANUP.PY

A small tool for log cleanup that removes different logs from spamming outputs. This cleans up our users experience and makes the overall UI experience cleaner.

```
import logging
from transformers.utils import logging as transformers_logging
def cleanLogs():
    logging.disable(logging.CRITICAL)

    # Hide Hugging Face / Transformers progress bars
    transformers_logging.disable_progress_bar()
```

Uses Python's standard logging system and utilities with Hugging face transformers to clean logs

TOOLS – TITLE.PY

Creates our header() method for the team 5 ASCII title at the start of the project

TOOLS – FOLDER_PATH_LOG.PY

Identifies folders and remembers previously selected locations in config.ini. It also finds report files inside folders and subfolders

CONFIGURATION STORAGE

```
CONFIG_FILE = "config.ini"
SECTION = "Settings"
DEFAULT_KEY = "last_folder"
```

The system will store previously selected documents and knowledge under separate keys in the Settings section of config.ini

```
def save_key_to_config(
    key,
    pair,
    section=SECTION,
    config_path=CONFIG_FILE
):
    config = load_config(config_path)

    if section not in config:
        config[section] = {}

    config[section][key] = str(pair)

    with open(config_path, "w") as f:
        config.write(f)
```

This method saves a folder without other settings already stored in config.ini

FOLDER SELECTION

```
def get_folder_path(
    key=DEFAULT_KEY,
    config_path=CONFIG_FILE,
    prompt_text=None,
    prompt_for_new_folder=False
):
    saved_folder = get_saved_folder(key, config_path)

    if not saved_folder or saved_folder == "None":
        folder = prompt_for_folder(
            prompt_text,
            prompt_for_new_folder=prompt_for_new_folder
        )
```

```
        save_key_to_config(
            key,
            folder,
            config_path=config_path
        )

        return folder
```

This is the main method used; it allows the user to reuse a previous folder or enter a new one with malware reports. It also validates that the folder exists and can create it when requested.

RECURSIVE FILE SEARCH

```
def list_files_recursive(folder_path: str) -> list[Path]:
    return [
        Path(root) / filename
        for root, _dirs, files in os.walk(folder_path)
        for filename in files
    ]
```

This method finds the files inside the selected folder and its subfolders. It allows reports to be organized by source while still being processed by the system.

TOOLS – AWS_KEY_MANAGER.PY

Allows user to enter, save, and validate the Pluralsight AWS credentials for Amazon Bedrock

ENTERING THE CREDENTIALS

```
def enter_new_keys(config_path="", aws_section={}):
    if config_path == "":
        config_path = get_default_config_file()

    config = load_config(config_path)

    access_key_id = getpass(
        "AWS access key: ",
        echo_char="*"
    ).strip()

    secret_access_key = getpass(
        "AWS secret key: ",
        echo_char="*"
    ).strip()

    region = input("AWS region: ").strip()
```

The getpass() function hides sensitive credentials while they are entered. If the credentials already exist, the system displays only the final four characters to see a preview

SAVING THE CREDENTIALS

```
config["AWS"] = {
    "access_key_id": access_key_id,
    "secret_access_key": secret_access_key,
    "region": region
}

save_config(
    config=config,
    config_path=config_path
)

print("Keys saved ✅")
```

The credentials and region are saved under the AWS section in config.ini. This keeps them outside the Python.
Because this file contains sensitive information, config.ini must NOT be committed to GitHub (.gitnore). Only trusted users should be able to view these credentials.

VALIDATING THE CREDENTIALS / REGION

```
session = boto3.Session(
    aws_access_key_id=aws_access_key_id,
    aws_secret_access_key=aws_secret_access_key,
    region_name=region_name
)

sts_client = session.client("sts")
identity = sts_client.get_caller_identity()

ec2_client = session.client("ec2")
ec2_client.describe_regions(
    RegionNames=[region_name]
)

print(f"✅ Region: VALID '{region_name}'")
return True
```

The validation method creates a boto3 session and sends a request using AWS Security Token Service to validate the credentials. It will return the caller's identity to ensure they are valid. Will also check to ensure that the AWS region is reachable.

ERROR HANDLING

```
except EndpointConnectionError:
    print("Region is invalid or unreachable.")
    return False

except NoRegionError:
    print("No AWS Region was provided.")
    return False

except NoCredentialsError:
    print("No AWS credentials were found.")
    return False
```

The method returns true when credentials are valid and False when they are missing, expired, incorrected, or configured with an invalid region. main.py will check this result before using NOVA.

SYSTEM – INDEXREPORT.PY

Loads the malware reports, assigns each report a unique ID, adds source metadata, and divides the extracted text into searchable chunks.

CREATE A REPORT ID

```
def CreateReportID(report: Path) -> str:
    report = Path(report)
    reportBytes = report.read_bytes()

    return hashlib.sha256(
        reportBytes
    ).hexdigest()[:16]
```

This method reads a report and creates a shortened SHA-256 hash from its contents. This acts as a report ID and helps distinguish reports between other reports with different contents.

LOADING REPORTS

```
for reportPath in listOfReportPaths:
    reportID = CreateReportID(reportPath)
    reportIDs.append(reportID)

    documents = SimpleDirectoryReader(
        input_files=[str(reportPath)]
    ).load_data()

    print(
        f"Documents loaded from "
        f"Report[{reportID}]: {len(documents)}"
    )
```

LlamaIndex's SimplyDirectoryReader loads each PDF and extracts its readable content. The generated report ID is saved for later identification.

ADDING METADATA

```
for document in documents:
    document.metadata.update({
        "reportID": reportID,
        "title": reportPath.stem.replace(
            "_", " "
        ).title(),
        "sourceFile": reportPath.name,
        "sourceOrganization": sourceOrganization,
    })
```

Metadata is attached to every loaded document before it is stored. This allows retrieved information to be connected back to its original report and organization if one is connected.

CREATING CHUNKS

```
splitter = SentenceSplitter(
    chunk_size=512,
    chunk_overlap=50,
)

chunks = splitter.get_nodes_from_documents(
    allDocuments
)

print(f"Chunks created: {len(chunks)}")
```

SentenceSplitter divides the reports into chunks of ~512 tokens. Each chunk overlaps the previously chunks by 50 tokens, so context is not lost between sections.

These chunks are later converted into embeddings and stored using ChromaDB.

RETURNING DATA

```
return reportIDs, allDocuments, chunks
```

The method returns 3 values:

- `reportIds` – unique identifiers for every report
- `allDocuments` – the complete documents loaded by LlamaIndex
- `chunks` – smaller sections that will be used for embedding later

SYSTEM – MODELS.PY

Configures the local BGE embedding model and NOVA used by the RAG system.

LOCAL EMBEDDING MODEL

```
def getEmbeddingModel():


    return HuggingFaceEmbedding(
        model_name="BAAI/bge-small-en-v1.5",
        device="cpu"
    )
```

This method loads the BAAI/bge-small-en-v1.5 embedding model through Hugging Face. The model runs locally on the VM's CPU and converts report chunks and inputted questions in vectors.

The model is downloaded automatically during its first use and is cached for future use sessions.

AMAZON NOVA MODEL

```
def getBedrockModelQueryEngine():
    model = "amazon.nova-pro-v1:0"
    print(f"Using Bedrock model: {model}")


    config = load_config_and_get_section(section="AWS")


    llm = BedrockConverse(
        model=model,
        temperature=0.0,
        max_tokens=8_000,
        aws_access_key_id=config.get("access_key_id"),
        aws_secret_access_key=config.get("secret_access_key"),
        region_name=config.get("region"),
    )


    return llm
```

This method loads the AWS credentials and the region from the config.ini file. Then it will configure the Nova Pro model through AWS Bedrock and returns the mode that is used for queries.

LOADING THE EMBEDDING MODEL

```
def get_embedding_llm_print():
    print("Loading embedding model...")
    embedding_llm = getEmbeddingModel()
    print(embedding_llm.model_name + " loaded ✅")
    return embedding_llm
```

Loads the embedding model and returns it for use.

SYSTEM – CHROMADB.PY

Creates, saves, and loads the ChromaDB knowledge base that will be used for storing the data

INITALIZING CHROMADB

```
def init_RAG_to_chromaDB(path = "storage/chroma"):
    projectRoot = Path(__file__).resolve().parent.parent
    databasePath = projectRoot / path

    db = chromadb.PersistentClient(path=str(databasePath))
    collection = db.get_or_create_collection(name="malware_chunks")
    vector_store = ChromaVectorStore(chroma_collection=collection)
    return vector_store
```

Creates the ChromaDB database and connects it with malware_chunks to store the Llamaindex collection.

SAVING EMBEDDING DATA

```
def save_RAG_to_chromaDB(chunks, embedding_model, path="storage/chroma"):
    vector_store = init_RAG_to_chromaDB(path)

    storage_context = StorageContext.from_defaults(
        vector_store=vector_store
    )

    index = VectorStoreIndex(
        nodes=chunks,
        storage_context=storage_context,
        embed_model=embedding_model,
        show_progress=True
    )

    return index
```

This method converts the chunks into embeddings and saves the chunks, vectors, and metadata into ChromaDB.

LOAD EXISTING EMBEDDING DATA

```
def load_RAG_from_chromaDB(llm, path = "storage/chroma"):
    vector_store = init_RAG_to_chromaDB(path)
    index = VectorStoreIndex.from_vector_store(vector_store, embed_model=llm, show_progress=True)
    return index
```

This method loads the exiting embedding data stored inside ChromaDB and connects the embeddings needed to search it

SELECT EXISTING EMBEDDING DATA

```
def load_embedding_data():
    folder = get_folder_path(key = "last_used_embedding_path", prompt_text="Please enter the folder path where your embedding data is stored: ")

    if folder is None:
        print("Failed to load embedding data. Valid folder not given.")
        return None

    path = Path(folder)

    with os.scandir(path) as it:
        if not any(it):
            print("Failed to load embedding data. Folder is empty.")
            return None

    embedding_llm = get_embedding_llm_print()

    index = load_RAG_from_chromaDB(embedding_llm, path=folder)
    return index
```

This method prompts the user to see if there is an existing folder with embedding/ChromaDB data stored from a prior session. It then checks to ensure there is data and then loads it if there is.

CREATE NEW EMBEDDING DATA

```
def create_embedding_data():
    rag_folder = get_folder_path(
        key="last_used_embedding_path",
        prompt_text=(
            "Please enter the folder path where you want to store your new embedding data: "
        ),
        prompt_for_new_folder=True
    )

    if rag_folder == None:
        return None

    documents_folder = get_folder_path(
        key="last_used_documents_path",
        prompt_text=(
            "Please enter the folder path where your documents are stored: "
        ),
        prompt_for_new_folder=False
    )
```

```
    if documents_folder == None:
        return None

    report_paths = [
        Path(path)
        for path in list_files_recursive(documents_folder)
    ]

    reportIDs, allDocuments, chunks = loadAndChunkReports(
        report_paths,
        ""
    )

    if reportIDs == None:
        print("Failed to load and chunk given Reports.\nReturning to options page...")
        return None

    embedding_llm = get_embedding_llm_print()

    index = save_RAG_to_chromaDB(
        chunks,
        embedding_llm,
        path=rag_folder
    )

    return index
```

This method selects the storage folder and the report folder and then processes those reports into chunks, creates the embeddings, and saves into ChromaDB.

SYSTEM – QUERYMANAGER.PY

Connects ChromaDB knowledge to Amazon Nova for questioning and prompting from the user

CREAT THE FINAL REPORT

```
def create_final_report(query_engine):
    topic = input(
        "\nWhat would you like the final report to be about?\n: "
    ).strip()

    if not topic:
        print("Please enter a report topic.")
        return
```

```
    # Default prompt used to generate the report
    prompt = f"""
Create a professional cybersecurity threat-hunting report about {topic}.

Use only information supported by the provided documents.

Include:
- Executive Summary
- Malware Overview
- Technical Behaviors
- Indicators of Compromise
- MITRE ATT&CK Techniques
- Know Exploitable Vulnerabilities
- Threat-Hunting Recommendations
- Detection Recommendations
- Sources

Do not invent unsupported information.
"""

    print("\nCreating final report...")

    try:
        response = query_engine.query(prompt)

    except Exception as error:
        print(f"\nReport generation failed: {error}")
        return

    project_root = Path(__file__).resolve().parent.parent.parent
    reports_folder = project_root / "reports"

    reports_folder.mkdir(
        parents=True,
        exist_ok=True
    )

    file_name = datetime.now().strftime(
        "final_report_%Y-%m-%d_%H-%M-%S.md"
    )

    report_path = reports_folder / file_name
```

```
    report_path.write_text(
        f"# Threat-Hunting Report: {topic}\n\n{response}",
        encoding="utf-8"
    )

    print("\n\033[1m| Final report created ✅ \033[0m")
    print(f"\033[1m| Saved to: {report_path}\033[0m")
```

This method asks the user for a prompt to give the model and sends it to the query engine and will save the response as a markedown file located in reports/

START THE QUERY ENGINE

```
def start_query_manager(index):
    # Connect the loaded RAG index to Amazon Nova
    query_engine = index.as_query_engine(
        llm=getBedrockModelQueryEngine(),
        similarity_top_k=5
    )

    print(
        "\n================================================================================"
        "\n\033[1m| I have loaded your documents. "
        "What would you like to know?\033[0m"
```

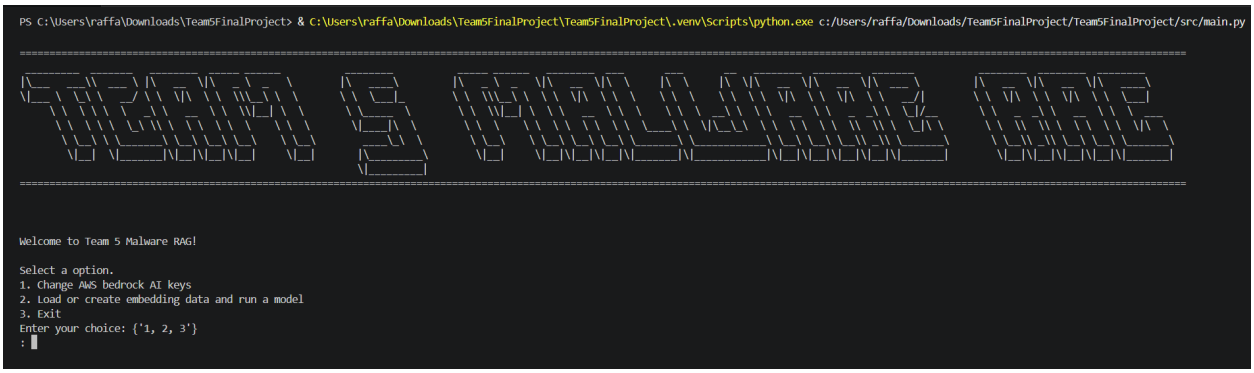
```
    )

    while True:
        query = input(
            "\nAsk a Question "
            "(Enter '2' to Create Final Report or 'back' to Return)...\n: "
        ).strip()

        if query.lower() == "back":
            print("\nReturning to the main menu.")
            return

        if query == "2":
            create_final_report(query_engine)
            continue

        if not query:
            print("\nPlease enter a question.")
            continue
```

```
        try:
            response = query_engine.query(query)
            print(f"\n\033[1m| {response}\033[0m")

        except KeyboardInterrupt:
            print("\n\nReturning to the main menu.")
            return

        except Exception as error:
            print(f"\nQuery failed: {error}")
```

This method will connect the loaded RAG index into Amazon Nova. It retrieves the most relevant chunks for each question prompted. Then, allows users to ask multiple questions and create the final report

USAGE

STARTING THE MAIN FILE (MAIN.PY)

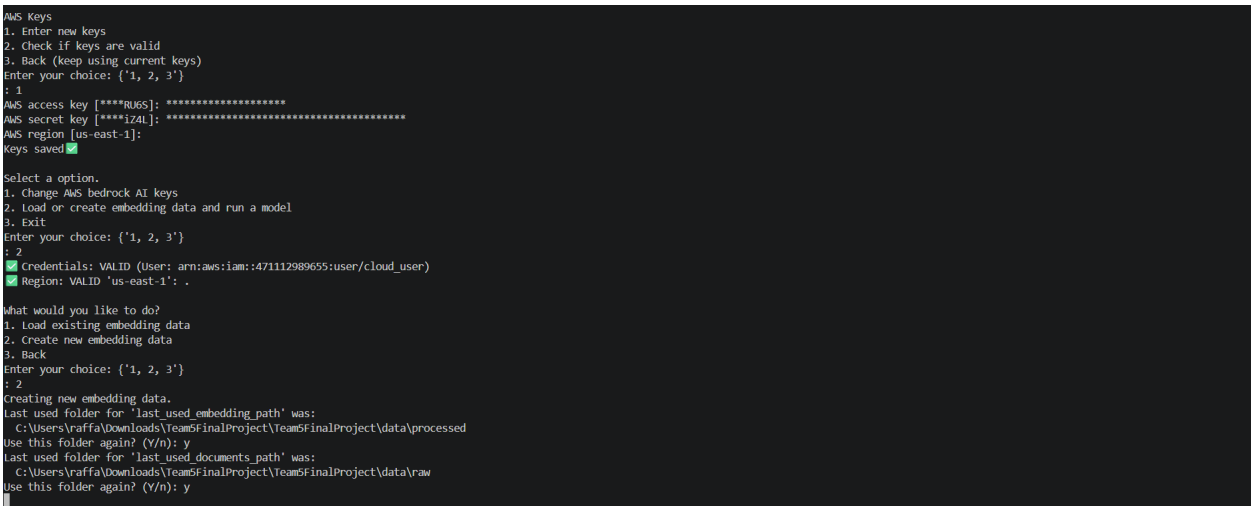


When the application starts, the main menu allows the user to update their AWS Bedrock credentials, load or create embedding data to begin using the RAG system, or exit the application.

ENTERING AND VALIDATING AWS/PLURALSIGHT KEYS

The AWS Keys menu allows the user to enter and validate their AWS Bedrock credentials. This example uses expired credentials, resulting in an InvalidClientTokenId error. Active credentials entered correctly would pass validation.

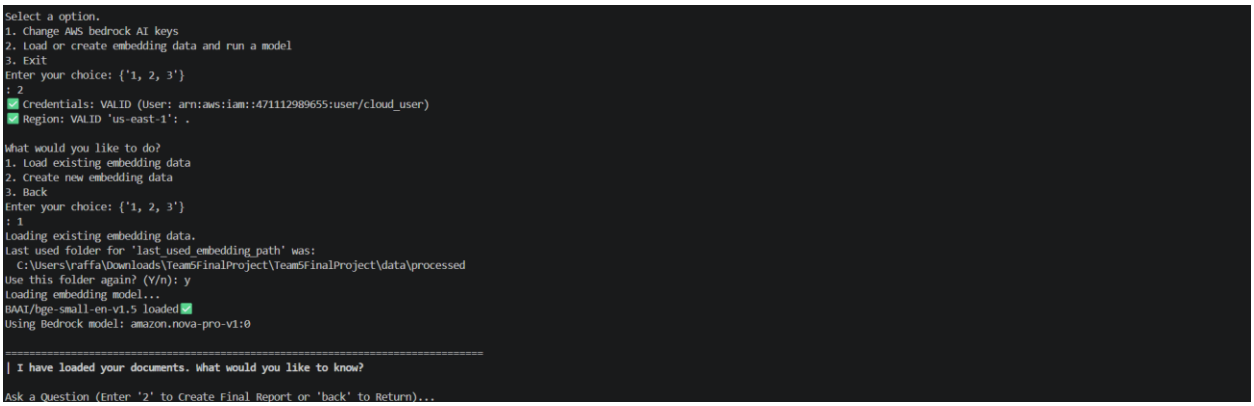
CREATE NEW EMBEDDING DATA



After validating the AWS credentials and region, the user selects the option to create new embedding data. The application then asks the user to confirm the folders used to store the embedding data and load the malware reports/documents.

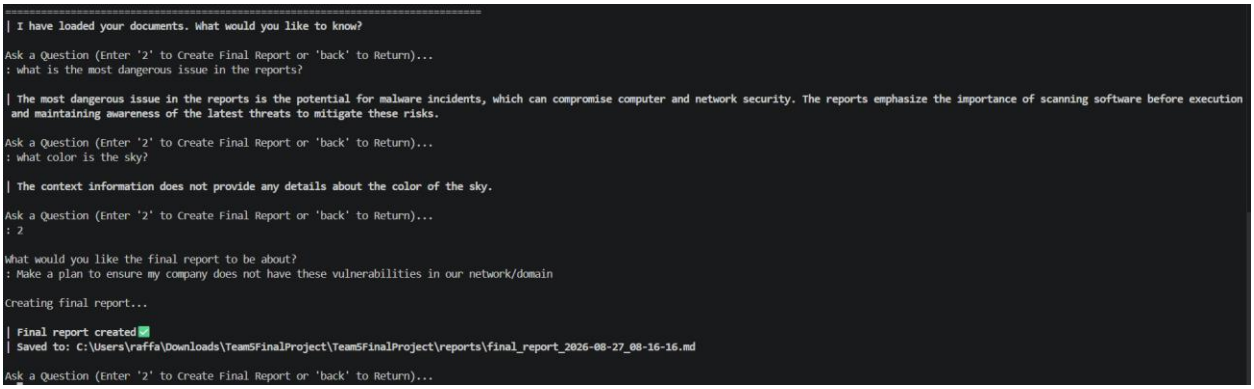
The application loads and divides the malware reports into chunks, then uses the local BGE model to generate and store their embeddings. Amazon Nova is then loaded and the user can ask questions or generate a final report.

LOAD EXISTING EMBEDDING DATA



The user can load previously created embedding data by selecting its saved folder. Then it loads the BGE embedding model and connects the stored data to Amazon Nova, allowing the user to begin asking questions without processing the original documents again.

ASKING QUESTIONS AND GENERATING FINAL REPORTS



The user can ask questions about the loaded malware reports, while unrelated questions are rejected when the documents do not provide supporting information. Selecting option 2 generates a final threat-hunting report and saves it as a timestamped Markdown file in the /reports folder.

ADDITIONAL NOTES

- When starting the system (main.py), it will take about a minute to start the system due to loading all the model data and reading through the database for existing malware data.
- Pluralsight AWS Sandbox credentials are temporary and must be replaced when they expire. The system is dynamic in that you can migrate from the Pluralsight environment and use another LLMS API instead.
- AWS credentials stored in config.ini must never be shared or committed to GitHub.
- AI-generated answers and reports should be reviewed by an analyst before being used for security decisions.
- Creating embedding data may take several minutes depending on the number and size of the documents. Previously created embedding data can be loaded to avoid processing the same documents again.